



CLAM: Cross-layer Adaptation Manager for Service-Based Applications

Asli Zengin
Fondazione Bruno Kessler
Trento, Italy
zengin@fbk.eu

Annapaola Marconi
Fondazione Bruno Kessler
Trento, Italy
marconi@fbk.eu

Marco Pistore
Fondazione Bruno Kessler
Trento, Italy
pistore@fbk.eu

ABSTRACT

The heterogeneous and dynamic execution context of service-based applications (SBA) makes the problem of adaptation critical. However in most cases the adaptation is not trivial due to the following facts. First, SBA has a complex layered system where the application is implemented through a composition of services, which in turn are provided by platforms and run on top of infrastructures. Second, as a result of this multi-level application system there exist several adaptation approaches isolated from each other, which focus on a specific concern of one level ignoring the overall impact of the adaptation on the whole service-based system. To tackle this problem we propose a cross-layer adaptation manager (CLAM) whose contribution is two-fold: (i) It provides a platform that integrates and coordinates existing analysis and adaptation tools, which target specific system concerns, to assess the impact of an adaptation at the different levels. (ii) Covering the whole system for the SBA, it provides an analysis algorithm that incrementally constructs consistent adaptation strategies starting from an initial adaptation trigger originated at any level. The paper introduces the proposed approach and presents its first implementation with concrete analysis and adaptation tools.

Categories and Subject Descriptors

C.2.4 [Computer Systems Organization]: Distributed Systems—*Distributed applications*; C.4 [Computer Systems Organization]: PERFORMANCE OF SYSTEMS—*Reliability, availability, and serviceability*; H.3.4 [Information Systems]: Systems and Software—*Performance evaluation (efficiency and effectiveness)*; K.6.4 [Computing Milieux]: System Management—*Quality assurance*

General Terms

Design, Management, Performance

Keywords

adaptation management, service-based applications, cross-layer adaptation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

QASBA '11, September 14, 2011, Lugano, Switzerland
Copyright 2011 ACM 978-1-4503-0826-7/11/09 ...\$10.00.

1. INTRODUCTION

The operation of service-based applications is a complex task. These distributed systems must provide their functionalities with the required/agreed qualities of services (i.e., service-level agreements), must cope with the unreliable network on which they operate, and also deal with the changes in the context in which they are executed, or in the partner services with which they interact. This means that the infrastructure must probe the execution to discover all these problems as soon as they materialize, and these applications must be able to adapt their behavior to cope with them.

Adapting the behavior may mean changing the actual composition of services, or selecting different partners services, but the satisfaction of the service level agreements in place may also impose changes in the way services are offered. For example the supervision system may re-negotiate some quality parameters with the providers of the partner services, change the configuration of the engine that runs the composition (BPEL process), tune the platforms that provide some partner services, and even adjust the infrastructure (resources) used by the application and its partners.

This is to say that a problem (*adaptation need*), which usually materializes at application level, may trigger adaptations at any level of the “usual” service stack [16]: software (application), platform, and infrastructure. Moreover, some adaptations may trigger others, or they may influence some quality parameters, or even the operation, of parts of the system. For example, the selection of a new cheaper service could help decrease provisioning costs, but if the new service were slower than the previous one, it would have a negative impact on the performance of the new application. Similarly, to keep the performance agreed with clients, the same application, or some partner services, may require the provision of additional resources to allocate all the different instances of the application and run them efficiently. Again, additional resources may increase the price of the application, and thus a viable solution must find a compromise between the two extremes.

Adaptation is thus a cross- and multi-layer problem, and at each layer it could also target different, maybe conflicting, *concerns*. For example, when reorganizing the composition, one could privilege the application’s price, its speed, or the compliance with some external regulations. Many existing solutions [14, 3, 12, 7, 13, 5, 15, 6, 11] have addressed adaptation in a “local” way by only considering one concern at one layer; in contrast this paper fosters a holistic approach able to address different layers and concerns concurrently, to reason on the dependencies and consequences among the different aspects, and to identify global solutions. These solutions must harmonize layers and concerns, and provide an integrated adaptation plan based on local activities.

The paper proposes a consistent and coherent solution for holistic adaptations based on CLAM (Cross-Layer Adaptation Man-

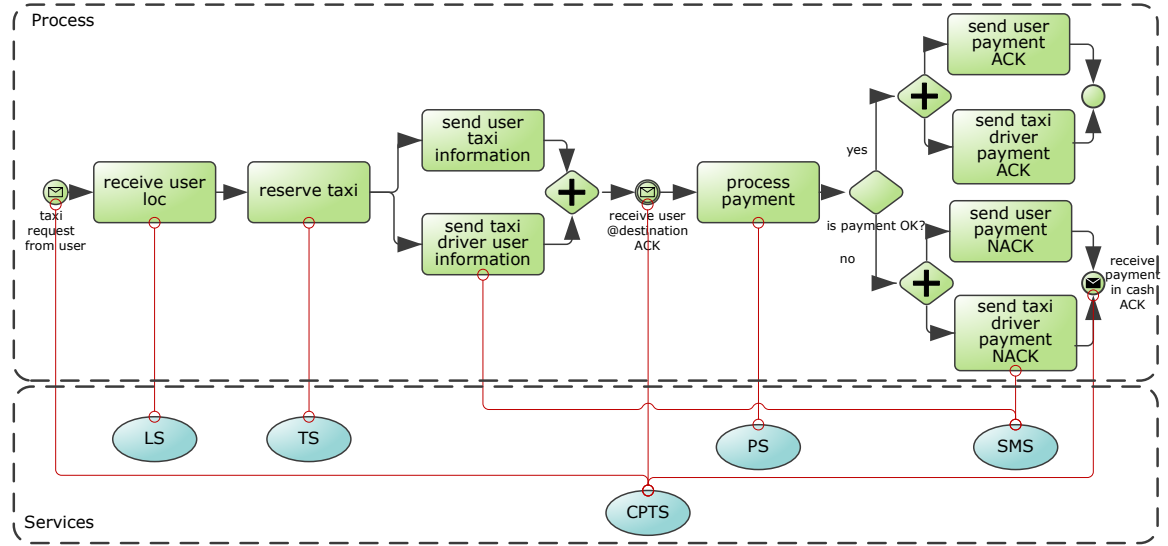


Figure 1: Call & Pay Taxi Application Model

ager). The manager is based on a comprehensive high-level model of the application and of the layers behind it. Each model element is associated with (i) a set of analyzers, specific to the different concerns, to understand the problem, (ii) a set of solvers, to identify possible solutions, and (iii) a set of enactors, to apply them on the element. The coordinated operation of analyzers, solvers, and enactors is governed by domain-specific rules that identify the dependencies, and consequences, between the elements of the model and run the different tools. For each adaptation need, CLAM produces a tree of the possible alternative adaptations, identifies the most convenient one, and applies it. The paper exemplifies all main concepts through a simple application for the smart management of taxi reservations.

The rest of the paper is organized as follows. Section 2 motivates and explains the need for holistic, cross-layer adaptations. Section 3 introduces the system representation for the proposed solution, Section 4 presents the overall approach and the system architecture behind it, and Section 5 presents a first implementation of the approach. Section 6 surveys some related approaches, while Section 7 concludes the paper.

2. PROBLEM STATEMENT

In this section we present a scenario to illustrate the cross-layer adaptation problem addressed in this paper (Figure 1). It will be used also to explain our approach in the next section.

We define a service-based system (SBS) as a system that is composed of an *application layer* where the service-based application takes place along with its non functional process properties such as key performance indicators (KPI), a *service layer*, which corresponds to the partner services of the process forming a platform for it, and finally the underlying *infrastructure layer* for the composite service (application) and the partner services. Our scenario is an SBS with the following layer characteristics:

- *Application layer*: “Call & Pay Taxi” composite service (CPTS), implemented as a BPEL process.
- *Service layer*: The partner services of our application, namely a short messaging service (SMS), a location service (LS) and a payment service (PS) provided by the telecom company,

and the taxi service (TS) provided by the taxi company.

- *Infrastructure layer*: The underlying platforms on top of which CPTS, SMS, LS, PS and TS run.

In CPTS, the client requests a taxi by sending a text message (SMS) to the application. Then, his/her location is identified and the taxi company is contacted to organize the real taxi service. After transporting the user to destination, the process terminates with a successful payment.

On this scenario we would like to show an adaptation case to motivate our problem. The CPTS provider decides to reduce the overall cost and consults the business analyst. Given that technologically it is already possible to replace services in the process, business analyst decides to switch to a cheaper telecom provider. This means replacing SMS, LS and PS in the BPEL process. However, there is a problem with the new LS service’s output message format. It provides client’s location in geographical coordinates instead of full address while in our application design we use full address as the input message to the taxi service. To address this data mismatch problem, the service composition is adapted by adding a mediator service in the workflow. Basically it converts geographical coordinates into the full addresses. Yet it triggers a new problem: the new service introduced for data mediation increases the overall cost of the process in an unforeseen way. The new workflow is functionally correct but the actual goal that led to the substitution of the telecom provider, that is cost reduction, is not met. In fact, the updated version of CPTS is more expensive than the original one.

Problem. With the existing approaches when an adaptation is performed, it targets a particular problem occurring at a specific concern of a specific SBS layer. Thus, they tend to propose local solutions to local problems in a way that is isolated from the overall application context. To avoid this, we must understand the impact of a change across different layers, which have their own characteristics and constraints. In our example while we are trying to improve the cost, one of the application key performance indicators (KPI), we do not know the consequences of replacing the services of the new telecom provider for the whole SBS. Consequently, we need an approach to meet the following requirements [20]:

R1. identify the problems that might occur in different concerns of the SBS layers due to an adaptation.

R2. if there exist problems, tackle them by proposing new adaptations in a consistency with the overall system.

We believe that addressing this problem is not trivial considering the complex, layered structure of the service based systems as well as the numerous existing analysis and adaptation approaches proposed and practised independently from each other. In next sections of this paper, we present CLAM that handles the complexity of the problem and meets the requirements enumerated above.

3. SBS REPRESENTATION FOR CLAM

For our CLAM approach, we need a proper representation of the service-based system that we introduced in the previous section.

In service-based systems, each layer has its own *elements* and each element has its own *concerns*. E.g. at the application layer while *KPI* and *process* are layer elements, *application execution time* is a concern related to the KPI element, and *data flow* is a concern related to the process element.

SBS's layered structure already contains the dependencies implicitly among different layer elements while those dependencies are not trivial considering adaptations coming from different layers. Hence, in our technique we benefit from the layer dependencies known for the system, and model the SBS in a cross-layer manner so that the dependencies get explicit to be used easily by our tool.

We follow a graph-like approach for the cross-layer modeling of the SBS, which we call "Cross-layer System Model" (Figure 2). In this model the graph nodes represent the system elements from different layers and the graph edges represent the cross-relations between these elements. We have "process", "process activity" and "KPI" elements for the application layer; "service provider", "service", "service operation" and "service QoS" elements for the service layer; and finally "infrastructure provider", "service infrastructure" and "infrastructure QoS" elements for the infrastructure layer.

The graph edges in the model display the dependency relation between different system elements. We distinguish two types of dependency relations: *has* and *consumes*. While *has* can only be a relation between two elements from the same layer; *consumes* defines the inter-layer relations whether an element from a layer relies on another element from a different layer. E.g. in infrastructure layer, the provider *has* the infrastructure and the infrastructure *has* the QoS attributes. On the other hand, since services are running on top of infrastructures, naturally service element from the service layer has a *consumes* relation with infrastructure element from the infrastructure layer. Similarly, for instance service availability depends on the infrastructure availability as well. Then, naturally there is a *consumes* relation between service QoS and infrastructure QoS.

To each system element we associate the set of relevant application concerns. For instance, attached to the process element we have the data flow concern (e.g. matching of exchanged message formats, satisfaction of data flow requirements) and the control flow concern (e.g. satisfaction of service composition requirements). CLAM allows to attach to each concern a set of adaptation capabilities that can be used to deal with its specific adaptation problems.

There are several state-of-the-art approaches for various types of SBS analysis and adaptation, and they already run in the system environment in an uncoordinated way. In our approach we would like to reuse those mechanisms to analyze the impact of an adaptation trigger and to extend it if required. Thus, we would like to include them in our "Cross-layer System Model". Each existing approach which can be used by our approach is called an *adaptation capa-*

bility.

We can have three different types of capabilities: (i) analyzers, (ii) solvers and (iii) enactors. While an analyzer checks the compatibility of a new adaptation for a system concern, a solver gets an incompatibility problem triggered by an analyzer and tries to propose an adaptation plan to handle the problem. After CLAM completes its overall analysis through a continuous interaction with analyzers and solvers, validated final adaptations are applied in the system through enactors. E.g. for the concern "process execution time" at *KPI* system element, we can attach a time analyzer. On the other hand, a process migrator is an enactor that can migrate the running process instances to a new process model that can be attached to the *process* system element.

Apart from "Cross-layer System Model", another important concept of the proposed approach is the "Instant System Configuration" M_i : an instantiation of the "Cross-layer System Model" with concrete system elements. E.g., for our reference scenario, CPTS, we can instantiate a system configuration with the current BPEL file in use for the execution, partner services SMS, LS, PS that are provided by the current telecom provider Vodafone, and the TS that is provided by RadioTaxi Trento. Similarly we can instantiate each graph node with concrete system elements and overall graph instantiation will be an M_i . We keep an M_i as a combination of an *Instant Process Model* for the application layer, an *Instant Service Model* for the service layer and an *Instant Infrastructure Model* for the infrastructure layer. Each of those models is an XML file keeping the concrete instances of the relevant system elements.

Before moving on to the next section we would like to mention that although CLAM needs as input the cross-layer representation of the application domain where main elements of the system are introduced as nodes of the graph and the edges correspond to the relationships among them, the CLAM algorithm to make the analysis is not domain specific. While in this paper we have the taxi reservation scenario, which is introduced as an SBS; CLAM is able to work with other application domains too, as long as the application system can be modeled by its elements and their dependencies to each other.

4. CLAM ARCHITECTURE

The system architecture of our approach, given in Figure 3, has two main parts: the core CLAM platform, implemented in Java, where the entire analysis is orchestrated, and the external capabilities that we plug-in to the platform.

CLAM core consists of the following main parts: *Cross-layer rule engine* where CLAM rules are implemented for the overall supervision of the execution, a number of *checkers* each of whom is responsible to act as an interface between CLAM and a capability that is plugged in the platform, *model updater* that takes care of updating instant system configuration whenever a new adaptation proposal enters CLAM, *tree constructor* where the results of the cross-layer adaptation analysis are continuously updated and kept in a tree-form structure, and lastly the *strategy ranker* where the final tree is traversed to output validated and ranked adaptation strategies, which are safe to be realized.

Cross-layer Rule Engine, implemented by using JBOSS Drools¹, is the fundamental part of the entire system. It consistently communicates with all the other parts of the CLAM, and through the execution of its rules it initiates, guides and completes an adaptation analysis. Its main functionality is to update the cross-layer analysis tree based on the exchanged information with the analyzers and solvers. Communication with those capabilities are through

¹<http://www.jboss.org/drools>

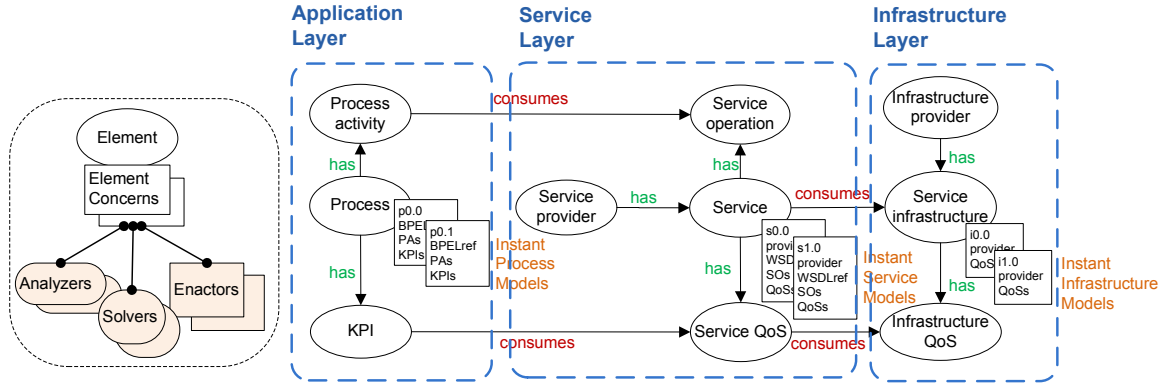


Figure 2: System Modeling for CLAM

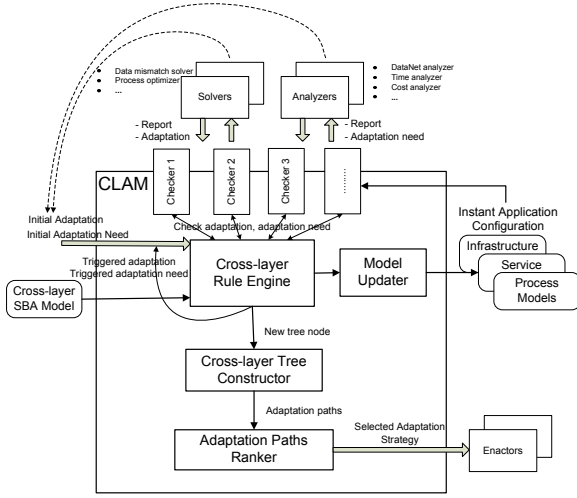


Figure 3: CLAM: Cross-layer Adaptation Manager

checkers since they are responsible for keeping track of the necessary input to be provided to an analyzer or a solver. Basically each capability produces a report as an output and the rule engine decides on the next step depending on the report type it gets from the component.

When the rule engine is activated due to an initial adaptation proposal or an adaptation need, a set of capabilities to be invoked are identified through a navigation of the affected system elements in the “Cross-layer System Model” and a prioritized queue of capabilities $Queue = \{C_1, C_2, \dots\}$ is created for this set. The queue is obtained considering the invocation priority of each adaptation capability, a property that is defined at design time, whenever a new capability is attached to the framework. E.g. we can say that a DataNet analysis at the process element is more emergent for us compared to a time or cost analysis at KPI element because, before analysing its performance, we want to make sure that an updated process is correct in terms of compositional requirements.

Checkers constitute an interface to contact a capability that is integrated with the CLAM platform. Whenever it is invoked by the rule engine, the checker first gets from the current system configuration the necessary data from the instant process, service and infrastructure models. Then, it contacts the requested adaptation capability by providing the required inputs. Upon receiving the result from the analyzer/solver, it prepares the report and sends it to

the rule engine. An analyzer can give three types of output: (i) the analysis is OK (the current system configuration is consistent with respect to the specific application concern); (ii) analysis is not OK and adaptation need is triggered (the current system configuration is not consistent but the problem can be solved through a new adaptation); (iii) Analysis is not OK (the system configuration is not consistent and the analyzer cannot suggest any new adaptation action). A solver can give two types of output: (i) Solution found, a set of alternative adaptation proposals is produced; (ii) Solution not found.

Model Updater. Whenever a new adaptation proposal enters the system due to an initial trigger or due to a solver report, the rule engine passes the adaptation information to the model updater so that the instant system configuration M_i can be updated. As well as sending the new system configuration M_i to the rule engine, the model updater also keeps track of all the versions of the instant process, service and infrastructure models. In this way, when a final adaptation strategy is decided by CLAM, the corresponding system configuration can be retrieved and realized in the SBS.

Tree Constructor. At each time a new analysis is initiated due to an initial trigger CLAM creates a new tree structure in order to keep track of the result of each capability invocation. We call it *CLAM Tree*, T , and each time CLAM receives back a report from a capability it adds an edge and a node to the T where a tree edge E_{tree} keeps the status of that report; a tree node N_{tree} keeps an updated instance of the system configuration M_i and an updated instance of the *Queue* of the capabilities to be invoked. CLAM continues to construct the tree until the overall analysis is completed.

Strategy Ranker. When the construction of a cross-layer adaptation tree is completed, next step is to extract the adaptation paths from the tree and to rank them based on our predefined criteria. Each path from the root node to a green leaf is an alternative adaptation strategy and for the time-being we rank those strategies based on their length. If a strategy contains more adaptations, it means more modifications for the system to be implemented, which is in most cases not preferable. In future work we will have more criteria for strategy ranking. E.g. we can give weights (costs) to different adaptation actions and select a minimum cost strategy. Moreover we can also consider aggregated QoS of the final instant process model for each strategy.

5. IMPLEMENTATION

We have a prototype implementation for CLAM. In this implementation we have five real tools (three analyzers and two solvers) from the state-of-the-art, plugged in the CLAM platform and CLAM

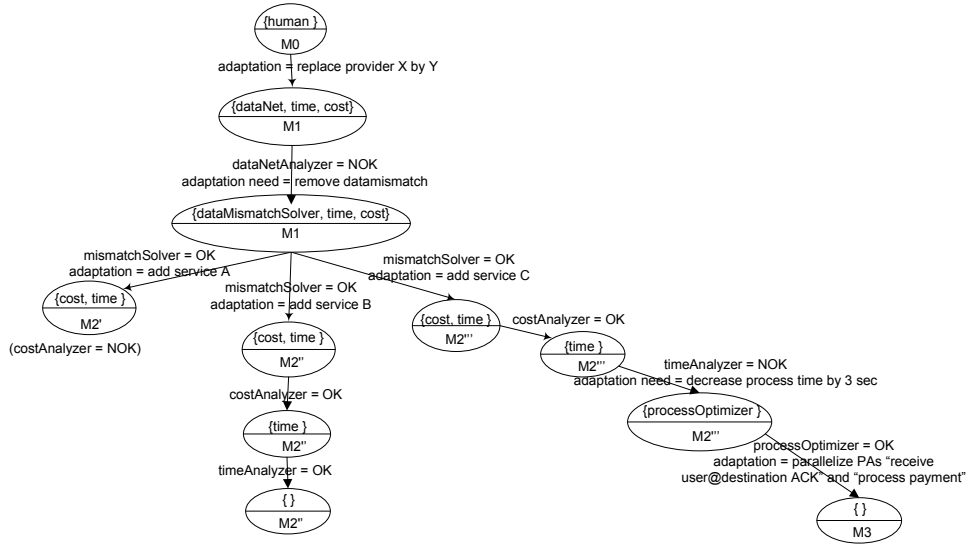


Figure 4: CLAM Tree for Alternative Adaptation Paths

coordinates them to produce a solution for the case study that we introduced in section 2

Below are the details of the tools that we used for our case study. However, our approach is not dependent on this case study, the implementation of CLAM is already extensible to accommodate new analyzers or solvers in the system.

QoS4BPEL is used both for time and cost analyses in our platform [8]. It gets as input the BPEL file of the application and the execution times and costs of each process activity in the BPEL, then it produces an aggregate value for them. Then those values can easily be compared with KPI target values to detect if a KPI is violated or not regarding a new process model.

Mismatch Patterns and Adaptation Aspects is used as both datanet analyzer and data mismatch solver [11]. In datanet analysis, whenever a new service is introduced to the composition, the compatibility of the new service interface is checked against the data flow requirements of the process. Mismatch Patterns and Adaptation Aspects tool gets as input the BPEL file and the WSDL file of the new service and then performs the analysis based on some predefined mismatch patterns for the application. If a mismatch is identified, the tool is also capable of producing an adaptor, i.e., a data mediator, which can be introduced to the composition as a service.

Structural Adjustment of BPEL is used as process optimizer to reduce the execution time of a process [18]. While in the case study we used it for time optimization, the tool is capable of optimizing the memory usage of the process as well. It gets as input the BPEL file and produces an optimized BPEL if any task parallelization is possible for the process.

The resultant tree that CLAM produces for our case study is given in Figure 4. When we receive the new adaptation proposal "replace telecom provider X by Y" from the business analyst (human), first CLAM checks which graph nodes are to be updated in the instant models. Then CLAM invokes the model updater asking to replace in the instant service models the old services with those of the new telecom provider (i.e. SMS, LS and PS). From the updated system configuration CLAM identifies that service provider, service, service operation and service QoS system elements are to be updated. This overall action implies an update in the instant system configuration (from M_0 to M_1). Then CLAM reasons about

the graph nodes that might be affected due to these changes in the configuration and finds out that process activity and KPI system elements might be affected since they consume the changed elements in the system. Next step is the decision of which analyzers to invoke in order to validate the telecom provider replacement. CLAM applies a principal rule for this purpose: It puts in the queue all the analyzers that are attached to the changed and affected graph nodes, in our example, DataNet analyzer that is connected to the process activity element, and time and cost analyzers that are connected to the KPI element in the cross-layer system model. After the creation of the queue, analyzers are invoked one by one and each time a report is received, tree is updated with the new tree edge and the tree node. If during the whole process, an analyzer reports back an adaptation need, then in this case, CLAM searches for a solver to be put in the queue, which can address the problem and produce an adaptation proposal. E.g. in the tree in Figure 4, DataNet analyzer reports back a datamismatch problem for the new LS, then CLAM discovers and invokes the MismatchSolver, and this capability produces three alternative mediator services (A, B, C) that can be added in the process to solve the data incompatibility problem.

After building the tree, we can select an adaptation strategy among alternative paths. Suppose that in the tree that we built for our scenario we select the rightmost path because the final application configuration in this path has the best QoS values. In this case the corresponding new application model would be as in Figure 5 where the updated system elements are highlighted in yellow, i.e., the new process activity that is responsible for data mediation and its new underlying mediator service DMS, the replaced TelCo services LS', PS' and SMS', our modified composite service CPTS' and finally the parallelized activities in the process.

The construction of the tree in Figure 4 took around 3 seconds on the 2 GHz Intel Pentium M Processor Windows XP laptop with 1 GB of RAM.

6. RELATED WORK

Existing work for SBA adaptation mostly focuses on specific aspects of the application where adaptation problems are solved merely in this narrow scope without taking into account the con-

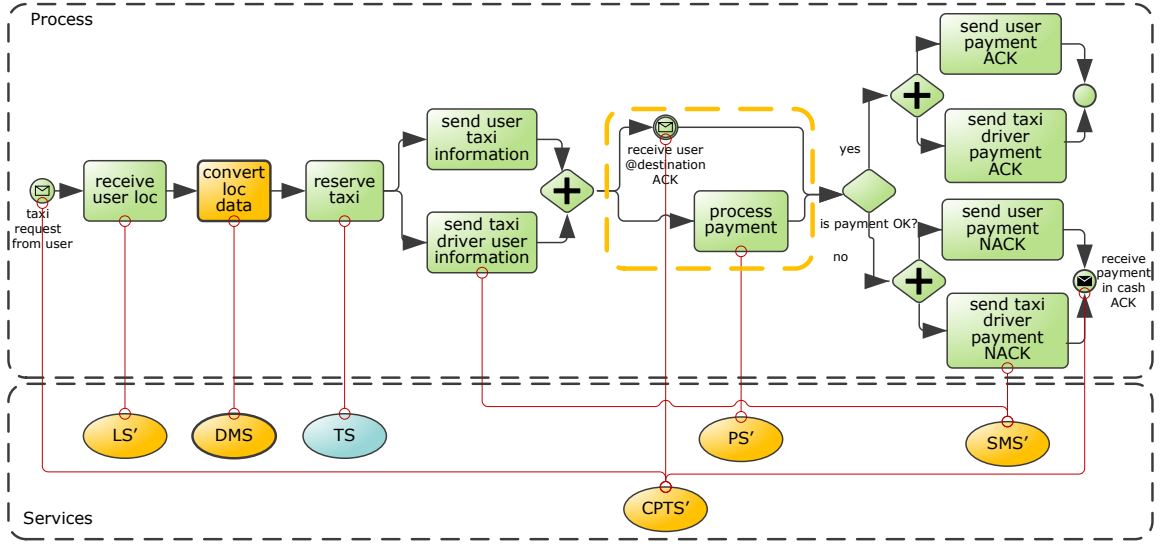


Figure 5: Call & Pay Taxi Application Model after CLAM Analysis

sequences for the whole SBA stack. BPM adaptation [14, 3], dynamic service binding [12, 7], self adaptation and self healing systems [13, 5], QoS-awareness [15, 6], mediator design for service interactions [11] and finally context-awareness [1] constitute these prevalent aspects. In principle these approaches might serve our platform as solver/analyzer capabilities and in this way they can be aligned with other SBA concerns and be more effective.

Yet, there are few approaches in literature that propose the use of some cross-relations for adaptation. In [9] coordinated adaptation is introduced for multiple applications interacting with each other in the same environment. These applications are not composed, but rather single entities which are affected by the same contextual attributes such as sharing common resources. The authors claim that a coordinated platform is important before adapting these applications to the context changes since they have interactions in the same environment. This problem is addressed in a narrow scope, only in terms of management of shared resources. Another problem they mention is conflict resolution for two adaptation mechanisms. However, they expect the user to perceive the conflict, and to modify the mechanisms accordingly.

The approach in [10] analyzes the dependencies of KPIs on process quality factors from different functional levels of an SBA such as QoS parameters, and then an adaptation strategy is decided to improve all the negatively affected quality metrics in the SBA. This work proposes a set of adaptation solutions for KPI violations through the consideration of the non-functional dependencies of SBA layers. However, the work is at preliminary stage and it is not clear based on what kind of rules they decide on which adaptation actions to perform in order to tackle a KPI violation.

Another approach is [17], which presents a framework for cross-layer adaptation of service oriented applications that comprise of organization, coordination and service layers. The application stack is similar to the SBA stack, which we introduce in this paper. They propose a technique where cross-layer adaptation designer prepares the taxonomies of adaptation mismatches, and later designs the adaptation templates, also known as patterns, that define generic solutions to tackle mismatches. Their technique directly models cross-layer adaptation templates, i.e., their dependencies are known from design time. Eventually they are like fixed patterns for cross-layer adaptation cases. Instead in our approach we model the ap-

plication in a cross-layer fashion and then discover the cross-layer adaptation paths on the fly through the coordination of available tools and mechanisms.

Like our CLAM approach, the works [2, 19, 4] have a cross-layer representation of the application model. While [19, 4] target limited number of adaptation cases such as service replacement, [2] makes use of the cross-layer model for monitoring and analysis rather than adaptation.

7. CONCLUSIONS AND FUTURE WORK

In this paper we have presented a holistic SBA management framework, CLAM, that can deal with cross- and multi-layer adaptation problems. This is achieved in two ways: on the one hand CLAM identifies the application capabilities affected by the adaptation actions, and on the other hand it identifies an adaptation strategy that solves the adaptation problem by properly coordinating a set of concern-specific adaptation capabilities. The proposed solution relies on a comprehensive high level system model that allows to easily extend the framework by plugging-in new adaptation capabilities. We implemented the proposed framework for the service-based application, taxi reservation.

In the future we plan to consolidate our implementation and evaluate it on application scenarios entailing higher level of complexity and requiring to deal with different system layers and concerns (e.g. Cloud-based applications).

8. ACKNOWLEDGMENT

The research leading to these results has received funding from the European Community's Seventh Framework Programme FP7/2007-2013 under grant agreement 215483 (S-Cube).

9. REFERENCES

- [1] D. Ardagna, M. Comuzzi, E. Mussi, B. Pernici, and P. Plebani. Paws: A framework for executing adaptive web-service processes. *IEEE Softw.*, 24(6):39–46, 2007.
- [2] L. Baresi, M. Caporuscio, C. Ghezzi, and S. Guinea. Model-Driven Management of Services. In *ECOWS'10*, pages 147–154, 2010.

- [3] A. Brogi and R. Popescu. Automated Generation of BPEL Adapters. In *ICSOC 2006: Service-Oriented Computing*, pages 27–39, 2006.
- [4] B. Burgstaller, D. Dhungana, X. Franch, P. Grunbacher, L. López, J. Marco, M. Oriol, R. Stockhammer, J. Universitat, and J. Universitat. Monitoring and Adaptation of Service-oriented Systems with Goal and Variability Models. Technical report, Universitat Politècnica de Catalunya, 2008.
- [5] A. Charfi, T. Dinkelaker, and M. Mezini. A plug-in architecture for self-adaptive web service compositions. In *ICWS '09: Proceedings of the 2009 IEEE International Conference on Web Services*, pages 35–42, Washington, DC, USA, 2009. IEEE Computer Society.
- [6] K. Christos, C. Vassilakis, E. Rouvas, and P. Georgiadis. Qos-driven adaptation of bpm scenario execution. In *ICWS '09: Proceedings of the 2009 IEEE International Conference on Web Services*, pages 271–278, Washington, DC, USA, 2009. IEEE Computer Society.
- [7] M. Colombo, E. D. Nitto, and M. Mauri. SCENE: A Service Composition Execution Environment Supporting Dynamic Changes Disciplined Through Rules. In *In ICSOC'06*, pages 191–202, 2006.
- [8] M. Dumas, L. García-Bañuelos, A. Polyvyanyy, Y. Yang, and L. Zhang. Aggregate quality of service computation for composite services. *Service-Oriented Computing*, pages 213–227, 2010.
- [9] C. Efstratiou, K. Cheverst, N. Davies, and A. Friday. An architecture for the effective support of adaptive context-aware applications. In *Proceedings of Mobile Data Management (MDM'01)*, pages 15–26, Berlin, January 2001. Springer.
- [10] R. Kazhamiakin, B. Wetzstein, D. Karastoyanova, M. Pistore, and F. Leymann. Adaptation of service-based applications based on process quality factor analysis. In *Service-Oriented Computing. ICSOC/ServiceWave 2009 Workshops*, pages 395–404. Springer, 2010.
- [11] W. Kongdenfha, H. Motahari-Nezhad, B. Benatallah, F. Casati, and R. Saint-Paul. Mismatch patterns and adaptation aspects: A foundation for rapid development of web service adapters. *IEEE Transactions on Services Computing*, pages 94–107, 2009.
- [12] A. P. S. J. A. M. Kunal Verma, Karthik Gomadam and Z. Wu. The meteor-s approach for configuring and executing dynamic web processes. Technical report, Department of Computer Science, University of Georgia, 2005.
- [13] M. F. L. Console. Ws-diamond: An approach to web services - diagnosability, monitoring and diagnosis, 2007.
- [14] L. T. Ly, S. Rinderle, and P. Dadam. Integration and verification of semantic constraints in adaptive process management systems. *Data Knowl. Eng.*, 64(1):3–23, 2008.
- [15] E. Nitto, M. Penta, A. Gambi, G. Ripa, and M. L. Villani. Negotiation of service level agreements: An architecture and a search-based approach. In *ICSOC '07: Proceedings of the 5th international conference on Service-Oriented Computing*, pages 295–306, Berlin, Heidelberg, 2007. Springer-Verlag.
- [16] M. Papazoglou, P. Traverso, S. Dustdar, and F. Leymann. Service-oriented computing: State of the art and research challenges. *Computer*, 40(11):38–45, 2007.
- [17] R. Popescu, A. Staikopoulos, P. Liu, A. Brogi, and S. Clarke. Taxonomy-driven adaptation of multi-layer applications using templates. In *SASO'10*, pages 213–222, 2010.
- [18] N. Rasadka and A. Marconi. Optimizing bpm compositions via automatic process re-writing. *Technical Report*, 2011.
- [19] U. Tripathi, K. Hinkelmann, and D. Feldkamp. Life cycle for change management in business processes using semantic technologies. *Journal of Computers*, 3(1):24, 2008.
- [20] A. Zengin, R. Kazhamiakin, and M. Pistore. Clam: Cross-layer management of adaptation decisions for service-based applications. In *2011 IEEE International Conference on Web Services*, pages 698–699.